union . graph_delete_version – удаление графа

union . destroy_revive_version – удаление/воскрешение union

union . union_element_update_version – обновление union методанных элемента

graphs . last_version

graphs . last_delete_version – удаление link-а

graphs . graph_element_update_version – обновление graph методанных элемента

НЕ ЭЛЕМЕНТА!

– А линки – это элементы или нет? ← НЕТ

почему нет? т.к. не нужны key, content. слишком сложно

– У нас есть Graph и Graph Element

Union и Union Element

• почему бы их не совместить и не сделать таблицы Graph и Union extend Element?

• тогда будут просто элементы разных видов и линки для графов – и не надо спец полиморфные передачи Union и Graph куда-нибудь

- и не надо лишние поля и join-ы по id_graph_element, graph_element_update_version, id_union_element, union_element_update_version.

- тогда key - это graph_name, union_name - эти поля уходят

- Graph "deleted" or "tombstone" flag.
(ч.б. active → tombstoned → delete_synced)

Теперь, что общая картина такая:

- Мы оптимистично отправляем только updates

- Если что-то удаляется, то мы томбстоним и отправляем томбстоне как update

- До того момента, как решаем сделать Sync
  - Element Sync
  - Graph Sync

- Тогда: graph last_delete_version — last_link_sync_version
  union graph_delete_version — last_graph_sync_version

Element Sync НУЖЕН, т.к. мы не хотим посылать Deletes для линков которые мы удалили - вдруг их нет.
НО, для графов UnionVersion это и есть Graph Sync!
т.е. Graph Sync не нужен, можно просто смотреть на версию и делать вывод из того, какие там графы

- таким же образом можно избежать Union Sync при мерже Family!

Однако при этом следует уделить особое внимание консистентной фильтрации на стороне бэкенда.

Так, бэкенд может запрашивать обновление по одному и тому же Union из нескольких источников.

Однако, бэкенд обязан использовать тот же самый фильтр для этого, и здесь лучше быстро зафейлить, если выяснится, что один и тот же Union запрашивается с разными фильтрами с разных источников.